

```

int main () {
int x = 7 , y = 20 , *m , *n , * k ;
int * arr ;

arr = ( int *) malloc ( MAX * sizeof ( int ) );

m = & x ;
n = m ;
k = & y ;
* m = 3;
* n = * k ;
*( arr ) = * m ;
*( arr +1) = * k ;
*( arr +2) = x + y ;

n = & y ;
* n += 5;

printf ( " %d , %d , %d , %d , %d , %d" , x , y , * arr , *( arr +1) , *( arr +2) , *m ) ;
return 0;
}

```

Paso 1 inicializa variables enteras

|   |       |
|---|-------|
| x | 0x100 |
|   | 7     |

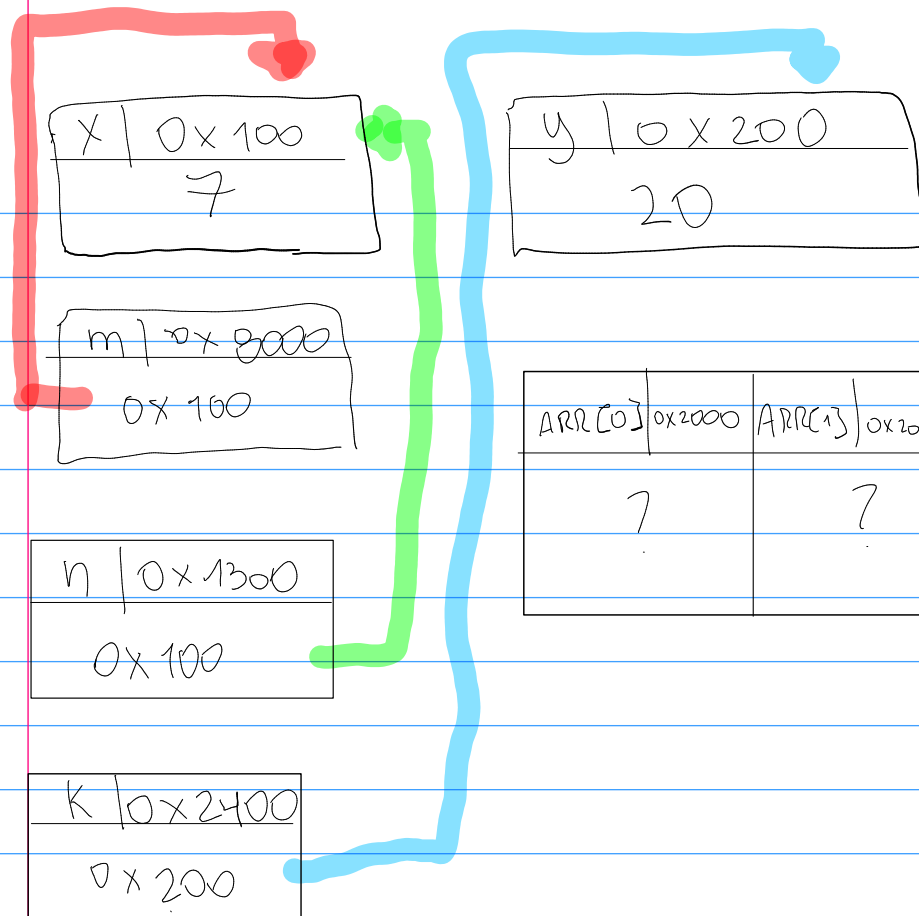
|   |       |
|---|-------|
| y | 0x200 |
|   | 20    |

|   |        |
|---|--------|
| m | 0x8000 |
|   | ?      |

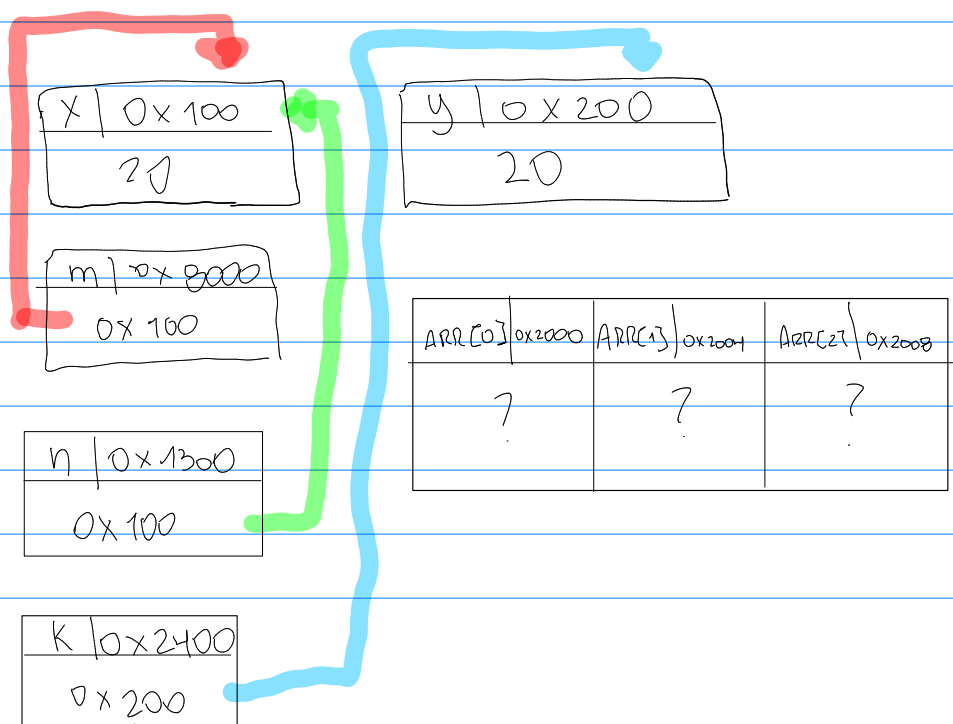
|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| ARR[0] | 0x2000 | ARR[1] | 0x2004 | ARR[2] | 0x2008 |
|        | ?      |        | ?      |        | ?      |

|   |        |
|---|--------|
| n | 0x1300 |
|   | ?      |

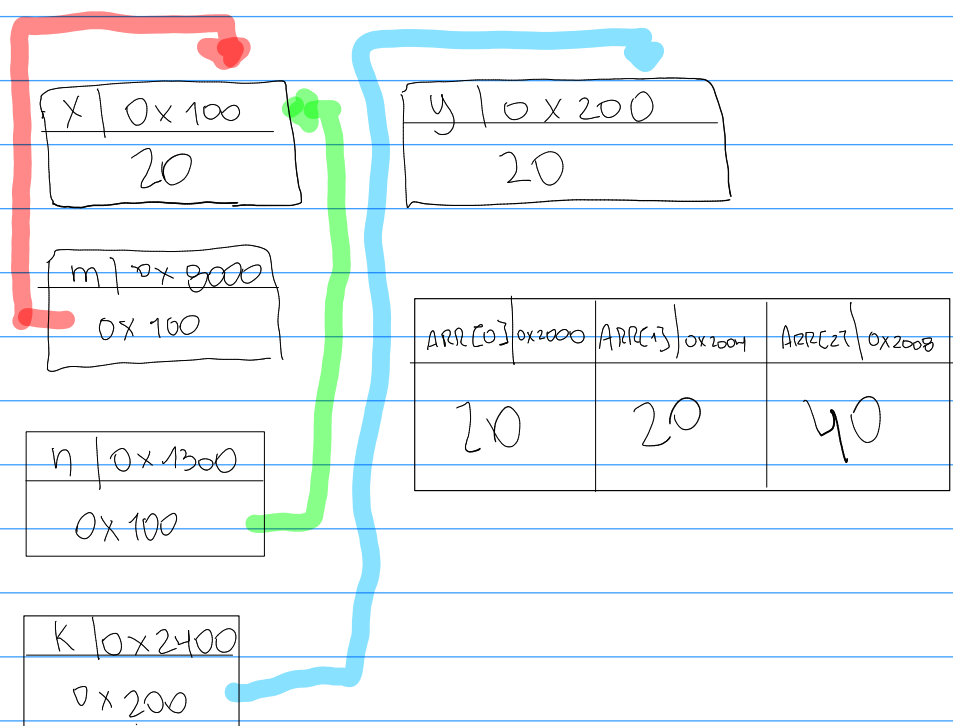
|   |        |
|---|--------|
| k | 0x2400 |
|   | ?      |



Puntero **m** se le asigna la dirección de memoria de **x**, al puntero **n** se le asigna el mismo valor ya que es igualado a **m**, el puntero **k** se le asigna la dirección de memoria de **y**.



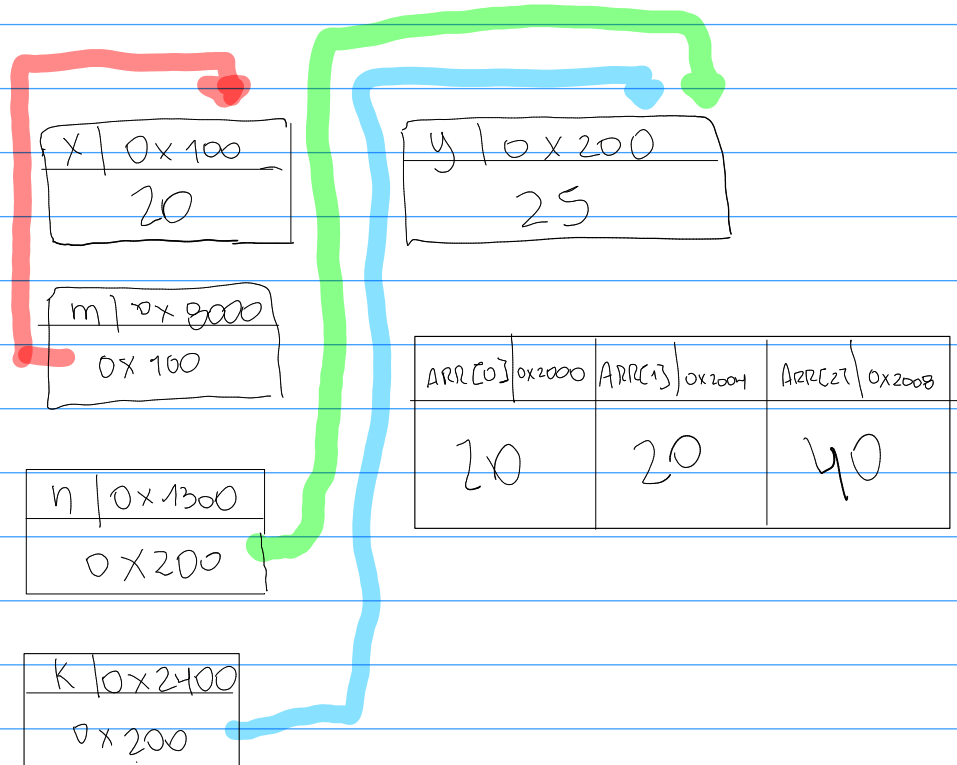
\*m = 3 entra a la dirección de memoria y asigna el valor 3, quedando  $x = 3$ , la siguiente línea  $*n = *k$  hace que \* entre a la dirección de memoria y le asigne el valor que hay dentro del puntero k, quedando  $x = 20$



\*Arr, accede a su valor y le asigna el valor que tiene dentro el puntero m que es 20, \*Arr+1 hace lo mismo pero le da el valor que tiene dentro el puntero k que es 20, a \*Arr+2 le asigna el valor de la suma de las variables  $x+y$ , quedando,  $20+20 = 40$ , asignando valor al puntero = 40

$n = \&y$  el puntero  $n$  ahora apunta a la dirección de memoria de  $y$

$*n += 5$ , accede a la dirección de memoria y le suma 5 quedando lo siguiente.



```
printf (" %d, %d, %d, %d, %d, %d", x , y , * arr , *( arr +1) , *( arr +2) , *m ) ;
```

Esta última línea imprime los valores quedando:

20 , 25 , 20 , 20 , 40 , 20